The Android Malware Static Analysis: Techniques, Limitations, and Open Challenges

Khaled Bakour,H.Murat Ünver,Razan Ghanem
Dept. of Computer Engineering
Kırıkkale University
Kırıkkale, Turkey
khaledbakour@kku.edu.tr, unver@kku.edu.tr, razan.ghanem86@gmail.com

***Abstract*— This paper aims to explain static analysis techniques in detail, and to highlight the weaknesses and challenges which face it. To this end, more than 80 static analysis-based framework have been studied, and in their light, the process of detecting malicious applications has been divided into four phases that were explained in a schematic manner. Also, the features that is used in static analysis were discussed in detail by dividing it into four categories namely, Manifest-based features, code-based features, semantic features and app's metadata-based features. Also, the challenges facing methods based on static analysis were discussed in detail. Finally, a case study was conducted to test the strength of some known commercial antivirus and one of the stat-of-art academic static analysis frameworks against obfuscation techniques used by developers of malicious applications. The results showed a significant impact on the performance of the most tested antiviruses and frameworks, which is reflecting the urgent need for more accurately tools.**

Keywords—Android; Static analysis; obfuscation; malware detection.

I. INTRODUCTION

Technology opened the doors to many applications and services that make our lives simpler, where, most of internet users is doing their shopping, browsing, organizing, bank transactions and monitoring business through smartphones. To meet the growing demand for electronic services the demand for smartphones is increasing dramatically over the time. According to Gartner's 2017 report [1] about the worldwide count of smartphones' sales, global sales of smartphones reached nearly to 380 million units in the first quarter of 2017 with 9.1 percent increase over the same period in 2016. Also, according to the same report, Google's Android extended its lead by capturing 86 percent of the total market in 2017 with 1.1 percent increase over points from a year ago. The main reasons behind the popularity of android can be shortened as, it is a free, flexible and open source OS, and its applications easy to build. Where, Google has provided tools that developers can use to build their applications, and it has provided the Play Store which is an official market that allows developers to upload their applications to it and making it available to the end-users. Furthermore, Google did not restrict downloading applications to android OS to be just from its official market but has maintained its openness in this area as well, to this end, Android OS allows downloading applications from third-party markets, where, there are many non-official markets which can be used freely to upload or download android applications.

On the other hand, the wide spread of Android and its open-source nature have made it a major target for malicious software developers. According to F-Secure's State of Cyber Security 2017 report [2] there are over 19 million malware programs developed especially for Android, making Android OS the main target for mobile malware. Additionally, according to the G DATA [3] Security Blog report for 2017, 750,000 malicious programs targeted at android have been discovered in the first quarter of 2017 only. Therefore, the security solutions that are proposed must be continuously updated to keep abreast of the evolution of the techniques followed by the developers of malware. The main contributions of this paper are:

More than eighty articles that used the static analysis and published between 2009 and the fourth month 2018 is studied from multiple sides. Android malware static analysis was explained in detail. The static features were classified into four type, namely, Manifest-based features, code-based features, semantic features, and app metadata-based features. The main challenges that is facing methods based on static analysis were discussed in detail. A case study was conducted to test the strength of some known commercial antivirus and one of the stat-of-art academic static analysis frameworks against obfuscation techniques used by developers of malicious applications.

II. ANDROID MALWARE DETECTION

We have divided the Android malware detection process in the light of the studied works into four phases, namely, Pre-processing phase, features extraction, features selection, and detection phase as illustrated in figure1. These phases will be discussed in detail in the next sections.

A. Pre-processing phase

In this stage, the dataset is prepared and processed to be in a suitable format to extract the features that will be used to generate the pattern that describes the behaviour of the application. There are three main methods used in this phase: static analysis, dynamic analysis and hybrid analysis. Since the

978-1-5386-7893-0/18/$31.00 ©2018 IEEE

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:58:34 UTC from IEEE Xplore. Restrictions apply.

subject of the article is static analysis, we will discuss the static analysis only in the four phases mentioned above.

Static analysis is the most widely used and preferred method by many researchers, because its computational time is low, it is easy to be implemented and it has proven its effectiveness to some extent in detecting harmful applications. In this way, the source code of the application is analysed without being executed in an emulator or in a real device. The first step is extracting an APK archive to retrieve the app's classes, manifest file, meta-data information and media files. In this step, the app's source code format is dex bytecode which not easy to be handled, so it can be de-compiled to java or smali code to make it readable and easy to be processed. To this end, multiple tools can be used, such as, Apktool, dex2jar, Ded, jd_gui, Dare and Androguard.

B. Features extraction phase:

As it has been mentioned before, in this paper only static analysis feature will be discussed. In general, this type of features depends on parsing the app's source code to extract the important information that can be used to form patterns to classify applications and detect any potential malicious behaviour. We will divide static features as follows:

1) Manifest based features: this file contains full information about the application, and when running any application, this file will be the first part to read, so it is like a roadmap that allow the program to work properly within the operating system. Due to the importance of this file, its data has been used extensively within the covered researches by this work. The features that are used in the covered researches can be classified as follows:

*1.1) Permissions:*A large number of the studied works depend on this feature, especially, the required permissions which are defined using <uses-permission> within Manifest file, such as in [5, 6][7]. In some other works a mapping between API calls and requested permissions has been used to avoid permissions' over privilege [8, 9]. In [10, 11], the used permissions with some app metadata such as, app's price, number of downloads, user rating, and description were used. Furthermore, in some works a combination of permissions and code-based feature such as, API, network addresses, intents...etc have been used [9].

1.2) Intent filter: it is a powerful feature that can be used to detect suspicious behaviour of applications where the intent filter can be used to describe the exact details of the intentions of the application. This feature was used in multiple works such as [12, 13].

1.3) hardware requirements: the app's hardware access requests (which is translated as a specific permission request in the Manifest file) have been used as a feature to detect malicious behaviour of applications in some works [13, 14].

1.4) Other manifest features: In some works, some manifest's file other information was used, such as activities, services, package name and Intents. For example, in [15] both of number of activities, number of services and number of receivers were used with some other code-based features.

2) Code-based features: These features can be extracted from variety of the source code's representation including opcode, bytecode, Smali and java code. We will classify the used code-based features as follows:

2.1) Instructions and commands features: The code is parsed to extract certain instructions that may describe the app's potential suspect behaviour. The following instructions-based features has been used in the studied works:

2.1.1) API calls: Android provides a wide range of APIs that can be used by developers to access operating system resources or hardware in the device. By preparing a suspicious API list and comparing it with the used APIs in the application the apps' suspicious behaviour can be revealed. API calls were used in a lot of works such as [14, 16] [17] [13], and in some works it have been used with their package level information and parameters such as in [18].

2.1.2)Specific instructions: some specific instructions and libraries' calls that frequently used by malicious apps are used as feature in previous works. For example, DexClassLoader, which is an API that can be used by attackers to download malicious content and execute it at the execution time, and Crypto API, which is library for coding the strings or other contents within the application have been used in [19]. In [16] other features such as Import terms was used. whereas, in [20], [21] some of dangerous Linux commands, such as using Su, Chmod and Exec has been used. Moreover, in [20] the app was checked for the presence of embedded .Dex, .Jar, .So, and .Exe files.

2.2) Strings and network addresses: The source code can be parsed to find any IP or DNS and use it for the program's behaviour identification, such as in [13][22]. Also, the strings in the source code of the application give a great indication to the application's behaviour and have been used in many previous studies such as, [11, 22, 23] [24] [25].

*3) Semantic features:*it indicates to any feature that have the semantic characteristics and includes code-based features, manifest file or even program description information-based features which are represented in a semantic way. the following are the most important used semantic features in the studied researches:

a) Control flow graphs: CFG is a directed graph represents all possible execution paths of the application. In [26], the same concepts of control graph are used to build the API call graphs and construct semantic signatures to detect unknown malware variants. Also, in [27] the control flow graphs are built based on native code for constructing semantic-based signatures can be used to detect malicious behaviour in both bytecode or native code.

b) Data dependency graph: A data dependence graph (DDG) is a common program analysis structure which represents inter-procedural flows of data within the app[28]. for example, in [28], a data flow analysis was used to construct a data dependency graph for user's inputs and API site calls and control flow analysis to reveal the Intent-based inter-app or inter-component events.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:58:34 UTC from IEEE Xplore. Restrictions apply.

c) Taint flow: In this type of analysis, a sensitive data that produced by an API is tracked from the source up to the target (source-sink). So, the source-sink data flow strings are extracted to use as apps' behaviour patterns. This technique is used in many of previous works such as, [29, 30].

d) Other semantic features: in some works, a semantic pattern was generated based on the app description and was compared with the actual behaviour of the app, such as in [30]. It should also be noted, that the source code sequence can be used as a semantic feature, like in [31] where, opcode n-grams was used to construct feature vectors that was used in classification of android apps.

4) Application's metadata-based features: we have classified the features related to app's descriptions or information as metadata features, including the following:

a) App certificate information: It includes the contents of the META-INF folder, which contains the signature of the application, certificate, and key which used to sign the app. This feature was used with some other features in [32] to detect the applications' re-packaging.

b) App' rating info in app store: it includes all app's evaluation information, such as price, number of downloads, user rating, and app description. It was used in [11] with some other descriptive information for App categorization.

c) Other descriptive features: Some other descriptive features were used in some researches, such as using APK's Hash value, application name and application icon in [32], and using number of lines in the manifest file and number of files and folders within APK file and its size in [19].

C. Features selection phase:

features selection is performed to reduce the dimensions of the features' data set by refining redundant or irrelevant features that can be lead to several problems such as, misleading the learning algorithm, reducing generality (overfitting), and increasing model complexity and run-time. So, the features are filtered according to their representative capacity for the entire data set. The most of feature selection algorithms which were used in the studied researches are feature ranking algorithms such as Information Gain, Mutual Information (MI), Chi-squared, and Relief. Etc.

D. Detection phase:

In this phase, the features which were chosen in the previous phase will be used to train a model to be used in the applications' classification process. In the most previous works these patterns are represented as binary vectors, where 0 represents feature's absence and 1 represents feature's presence. It is possible to use labeled or unlabeled data according to way that they will be used in the detection phase. In general, there are two approaches used in the researches that covered in this research, namely, signature-based and machine learning based. In Signature-based method, apps' patterns are generated and stored in a signature database, and unknown apps will be compared with these patterns to detect its behaviors. Generally, this method fails to detect unknown or polymorphic malware or zero-day attacks.

On the other hand, machine learning methods is introduced to handle the limitations of signature-based and the huge number of polymorphic malware and the new technologies of malware development. Through the researches that have been studied within this work, the bulk of the studies have used supervised learning algorithms, such as, SVM (Support Vector Machine), Naive Bayes, Decision Tree, Random Forests, K-Nearest Neighbours, and ensemble classifiers. A variety of this algorithms and methods have been used in the works that covered in this paper, such as [13, 14, 18, 33-36]. It is worth mentioning that, a small part has use some unsupervised clustering algorithms such as the k-means algorithm. Such as in, [5, 17, 25, 37, 38].

It should be noted that, N-fold cross-validation with N = 10 was used in the most of proposed methods. And the machine learning algorithms were implemented using WEKA tool in the most studies, in addition to using Scikit-learn Python library in some studies.

III. THE STATIC ANALYSIS LIMITATIONS AND CHALLENGES

Although static analysis is a lightweight detection method can detect malware on Android quickly with low computation complexity and fairly high performance, but it is still facing some problems.

Where the malware developers have begun using many techniques to hide app's malicious behavior, which has become a major challenge for static analysis methods. Through the study which has been conducted in this paper, we have seen that a few of the proposed frameworks tried to address some of these techniques. Thus, these techniques are considered as a big limitation of this type of analysis techniques. The most important of these techniques have been discussed in the following sections:

A. Obfuscation techniques

The code obfuscation changes the size and content of the Apk file, but without modifying the logical behavior of the app [39]. Some of obfuscation techniques were discussed below:

1) Name Obfuscation: in this method the package name or some other expressions in the code such as class names or methods names are changed to skip some analysis methods that use simple or descriptive features.

2) Control Flow Obfuscation: The call-graph of the app can be manipulated to defeat CFG based detection methods, this can be achieved by using Goto instruction to manipulate the sequence of instructions in the methods (Goto-obfuscation) [39, 40].

3) Class Encryption: it is an advanced obfuscation method, in which the entire class is encoded, compressed and stored in a data array, and a method is created to decrypt and load this class at run time, such that obfuscated class will be decrypted, decompressed, and then loaded in memory [41].

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:58:34 UTC from IEEE Xplore. Restrictions apply.

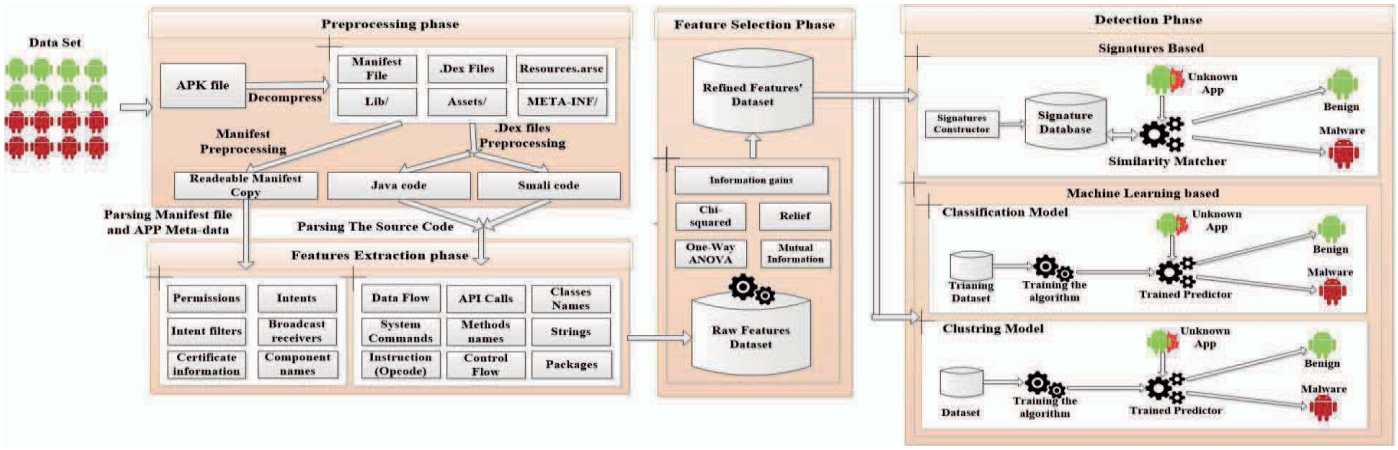


Fig. 1. Static analysis process phases

4) Strings Encryption: The developer of malicious code performs string encryption within the application to defeat strings-based detection techniques, where the original strings will be never found in the code, but its encrypted version which will be decrypted only at the execution time [39] .

5) Reflection: by using this technique, the class invocation or class's new instance creation can be achieved using literal strings to obfuscate the code and make it harder to be analyzed.

6) Junk code insertion: A junk code can be injected into the instructions of the classes and methods so that can be executed and does not affect the execution of the application.

7) Native code execution: The malware developers can write the malicious part of the code within native libraries to make application analysis more difficult. Where, a native code analyzing is considered much harder than bytecode analyzing, so a very few previous studies focus on the analysis of the native code [42].

B. Dynamic Payload:

The Android system allows the applications to load dex or jar files and execute it at the execution time. So, the malware developers can embed the malicious content within an encrypted dex or jar file. Furthermore, the malicious content can be downloaded from a remote server at the execution time, which, making it impossible to be detected using static analysis.

C. Applications repackaging:

Simply, app repackaging based on decompiling one of popular applications, adding the malicious content to it, re-compiling it and signing it with a new certificate.

Table I summarizes the studied researches and clarifies whether the challenges which facing the used analysing methods have been addressed or not.

IV. CASE STUDY

The goal of this case study is to test the anti-malware tools' robustness against obfuscation techniques. To this end, several famous commercial antiviruses, namely, Avast, Bitdefender, F-secure, G Data, and Symantec have been used through a VirusTotal website. VirusTotal is an online service that analyzes submitted malicious content using more than 60 antivirus engines. Malpat [43] was also used in this study, which is a state-of-art academic static analysis framework based on permission-API correlation and random forests classifier and is publicly available online. It is worth mentioning that, we have been decided to use this tool just to its online availability and not for any other considerations.

A. Dataset

A Malgenom dataset [89] was used in this study, which is a malignant data set consisting of 1260 samples from 49 different family.

TABLE.I. THE WORKS THAT COVERED IN THIS PAPER

#	Tool name	Year	Used features	Feature selection	Detection method	Countering malicious techniques			
						Rep	*Nat*	*Dyn*	*Cryp*
1	DroidDet [32]	2018	Mani_F, cod_F	TF-IDF, cosine similarity	Machine learning	×	×	×	×
2	Gurulian.et al [44]	2018	Meta_F	-	Signature Similarity	✓	×	×	×
3	KUAFUDET [21]	2018	Cod_F, Semtc_F	Manual pruning	ML+ Similarity	✓	×	×	×
4	SAFEDroid [19]	2018	Cod_F, Mani_F, Meta_F	-	Machine learning	×	×	✓	✓
5	Chao .et al [45]	2018	Mani_F, Semtc_F	-	Pattern mathing	×	×	×	×
6	Wei .et al [46]	2018	Cod_F, Mani_F	Deep learning	Machine learning	×	×	×	×
7	Abubaker.etal[47]	2018	Mani_F	-	Machine learning	×	×	×	×
8	DroidFusion [20]	2018	Cod_F, Mani_F	Information gain (IG)	Machine learning	×	×	✓	×
9	Kirubava.et al [48]	2018	Mani_F	Information gain	Machine learning	×	×	×	×

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:58:34 UTC from IEEE Xplore. Restrictions apply.

10	MalPat [43]	2018	Cod_F, Mani_F	-	Machine learning	×	×	×	×
11	FalDroid [49]	2018	Semtc_F	TF-IDF-like approach	Machine learning	×	×	×	×
12	MOCDroid [16]	2017	Cod_F	Genetic Algorithm	ML+ Similarity	×	×	×	×
13	AndroDialysis [12]	2017	Mani_F	-	Machine learning	×	×	×	×
14	Wanga.et al [13]	2017	Cod_F, Mani_F, Meta_F	MuInfo,Chi2,ANOVA	Machine learning	×	✓	✓	✓
15	Pındroid [50]	2017	Cod_F, Mani_F	-	Machine learning	×	×	×	×
16	DroidNative [27]	2017	Semtc_F	-	Semantic-signatures	×	✓	×	×
17	Sokolovaa.et al [51]	2017	Semtc_F	-	Machine learning	×	×	×	×
18	Yao.et al [36]	2017	Semtc_F	-	Machine learning	×	×	×	×
19	Palumbo.et al [23]	2017	Cod_F , Meta_F	Threshold based	Machine learning	×	×	×	×
20	Yang.et al [52]	2017	Meta_F	Information gain	Signature based	×	×	×	×
21	Milosevica.et al [37]	2017	Cod_F, Mani_F	-	Machine learning	×	×	×	×
22	MalDozer [53]	2017	Cod_F	Deep learning	Machine learning	×	×	×	×
23	Dighe.et al [7]	2016	Mani_F	Gain Ratio, Relief, Cfs	Machine learning	×	×	×	×
24	Dexteroid [29]	2016	Cod_F	-	Pattern matching	×	×	×	×
25	Brown.et al [30]	2016	Semtc_F	-	Machine learning	×	×	×	×
26	Ma.et al [54]	2016	Cod_F, Mani_F	-	Machine learning	×	×	×	×
27	ANASTASIA [55]	2016	Cod_F, Mani_F	Extra Trees-Classifier	Machine learning	×	×	×	×
28	Coronado.et al [56]	2016	Cod_F, Mani_F	Chi2,Relief	Machine learning	×	×	×	×
29	SafeDroid [57]	2016	Cod_F	-	Machine learning	×	×	×	×
30	ASE [58]	2016	Mani_F, Meta_F	-	Threshold based	×	×	×	×
31	MAMADROID [59]	2016	Semtc_F	PCA	Machine learning	×	×	×	×
32	Verma.et al [38]	2016	Meta_F	Information gain	Machine learning	×	×	×	×
33	Wu.et al [60]	2016	Semtc_F	-	Machine learning	×	×	×	×
34	Elish. et.al [28]	2015	Semtc_F	-	Rules- based	×	×	×	×
35	APK Auditor [5]	2015	Mani_F	-	Signature-based	×	×	×	×
36	Yerima. et.al [61]	2015	Cod_F, Mani_F	Mutual Information	Machine learning	×	×	×	×
37	Canfora. et.al [31]	2015	Semtc_F	Mutual Information	Machine learning	×	×	×	×
38	Lantz. et.al [42]	2015	Cod_F	-	Pattern matching	×	✓	×	×
39	Qi Li [62]	2015	Cod_F	-	Pattern matching	×	×	×	×
40	Westyarian. et.al [63]	2015	Cod_F	-	Machine learning	×	×	×	×
41	Li. et.al [64]	2015	Cod_F, Mani_F	-	Machine learning	×	×	×	×
42	Gordon. et.al[65]	2015	Cod_F	-	-	×	✓	×	×
43	Sheen. et.al [35]	2015	Cod_F	IG, Relief, Chi-Square	Machine learning	×	×	×	×
44	Lee. et.al [24]	2015	Semtc_F, Cod_F	-	Signature-based	×	×	×	×
45	DENDROID [66]	2014	Semtc_F	-	Machine learning	×	×	×	×
46	Moonsam. et.al [9]	2014	Cod_F, Mani_F	-	Machine learning	×	×	×	×
47	DroidSIFT [67]	2014	Semtc_F, Cod_F	-	Machine learning	×	×	×	×
48	Drebin [14]	2014	Cod_F, Mani_F	-	Machine learning	×	×	×	×
49	Yerima. et.al [68]	2014	Cod_F, Mani_F	-	Machine learning	×	×	×	×
50	Manilyzer [34]	2014	Mani_F	-	Machine learning	×	×	×	×
51	Apposcopy [69]	2014	Semtc_F	-	Signature-based	×	×	×	×
52	Xiaoyan. et.al [70]	2014	Mani_F	PCA algorithm	Machine learning	×	×	×	×
53	XIANGYU [70]	2014	Mani_F	IG, Fisher Score, Chi2	Machine learning	×	×	×	×
54	Liang. et.al [71]	2014	Mani_F	-	Rule-based	×	×	×	×
55	MIGDroid [72]	2014	Cod_F	-	Graph similarity	✓	×	×	×
56	Idrees [73]	2014	Mani_F	-	Machine learning	×	×	×	×
57	Raphael. et.al [74]	2014	Cod_F, Mani_F	X–ANOVA, X–Utest	Graph similarity	×	×	×	×
58	DroidGraph [26]	2014	Cod_F	-	Semantic-signatures	×	×	×	×
59	Hosseini. et.al [15]	2014	Cod_F, Mani_F	-	Machine learning	×	×	×	×
60	Shen. et.al [75]	2014	Cod_F, Mani_F	heuristic based	Graph similarity	×	×	×	×
61	DroidAPIMiner [33]	2013	Cod_F, Mani_F	-	Machine learning	×	✓	✓	×
62	Samra [10]	2013	Mani_F	-	Machine learning	×	×	×	×
63	AppGuard [25]	2013	-	-	Policy based	×	×	×	×
64	A3 [76]	2013	Semtc_F	-	Pattern detection	×	×	×	×
65	Aung. et.al [77]	2013	Mani_F	Information Gains	Machine learning	×	×	×	×
66	DroidOLytics [78]	2013	Semtc_F	Similaritydigest hashing	Signature-based	✓	×	×	×
67	Peiravian. et.al [79]	2013	Cod_F, Mani_F	-	Machine learning	×	×	×	×
68	Glodek. et.al [80]	2013	Cod_F, Mani_F	-	Machine learning	×	✓	×	×
69	Zulie. et.al [81]	2013	Mani_F	-	Machine learning	×	×	×	×
70	AnDarwin [82]	2013	Semtc_F	-	Pattern matching	✓	×	×	×
71	DroidAPIMiner [18]	2013	Cod_F, Mani_F	-	Machine learning	×	×	×	×
72	AndroidLeaks [8]	2012	Cod_F, Mani_F	-	-	×	×	×	×
73	Sanz. et.al [11]	2012	Cod_F, Mani_F, Meta_F	Information Gains	Machine learning	×	×	×	×
74	DroidMat [17]	2012	Cod_F, Mani_F	-	Machine learning	×	×	×	×
75	DroidMOSS [83]	2012	Cod_F, Meta_F	fuzzy hashing	Signature-based	✓	×	×	×
76	RiskRanker [84]	2012	Semtc_F	-	Signature-based		✓		✓
77	Stowaway [86]	2011	Cod_F, Mani_F	-	Perm-API mapping	×	×	×	×

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:58:34 UTC from IEEE Xplore. Restrictions apply.

78	ComDroid [87]	2011	Cod_F, Mani_F	-	-	×	×	×	×
79	Barrera. et.al [6]	2010	Mani_F	-	Machine learning	×	×	×	×
80	Kirin [88]	2009	Mani_F	-	Rules -based	×	×	×	×

Cod_F: Code based features, Semtc_F: semantic features, Mani_F:Manifest features, Meta_F: app metadata features, ML:Machine learning
Rep: Repakaging, Nat: Using native code, Dyn: Dynamic code loading, Crypt: Encryption techniques

Five data sets, each of which contains 49 samples were used, the first one 49 samples were selected randomly from Malgenom. In the other datasets, the same chosen samples in the first group was used but after applying different obfuscation techniques on it. To this end, the PRAGuard data set [41] has been used. The used datasets details were shown in table II.

TABLE.II. USED DATASET

D1: Original dataset, i.e. without any obfuscation.
D2: Reflection dataset, i.e. statement invocation reflection method was applied.
D3: String encryption, i.e. The classes' strings was encrypted using XOR.
D4: Class encryption, i.e. the classes was completely encrypted and compressed.
D5: Full obfuscation, i.e. all previous techniques was used together.

B. Results

Figure 2 shows the results obtained from the case study, where the accuracy value for each type of used antiviruses was calculated for each dataset. It can be noted from the obtained results, the great decreasing of the performance of most of the antiviruses gradually when the complexity of the used obfuscation method was increased. For example, the use of Malpat's accuracy decreased from 71% when no obfuscation was used to 61% when the full_obfuscation dataset was used.

It is worth mentioning that, the accuracy value of Malpat when the full_obfuscation dataset was used is more than when the class encryption dataset was used. This can be explained by that Malpat could not recognize some of the files within the class encryption dataset ('Invalid APK' message) and these files were considered undetected and this leads to decrease the accuracy.

V. CONCLUSIONS

Static analysis is one of the most widely used and preferred methods by the vast majority of researchers, thanks to its ease of application, low overhead and good performance in detecting malicious applications to some extent. In this research, we have discussed this type of analysis in order to highlight its weaknesses and the challenges facing it. To this end, more than 80 static analysis-based framework have been studied, and in their light, the process of detecting malicious applications has been divided into four phases that were explained in a schematic manner. Also, the features that is used in static analysis were discussed in detail by dividing it into four categories: Manifest-based features, code-based features, semantic features and app's metadata-based features. Also, the challenges facing methods based on static analysis were discussed in detail, where these challenges are represented as the techniques that used by developers of malware applications to defeat static analysis frameworks. Finally, a case study was conducted to test the strength of some known commercial antivirus and one of the stat-of-art academic static analysis frameworks against obfuscation techniques used by developers of malicious applications. The results showed a significant impact on the performance of the most tested antiviruses and frameworks, which is reflecting the urgent need for more accurately tools that can make a deeper analyze to cope with the rapid increasing in the new malicious code obfuscation methods.

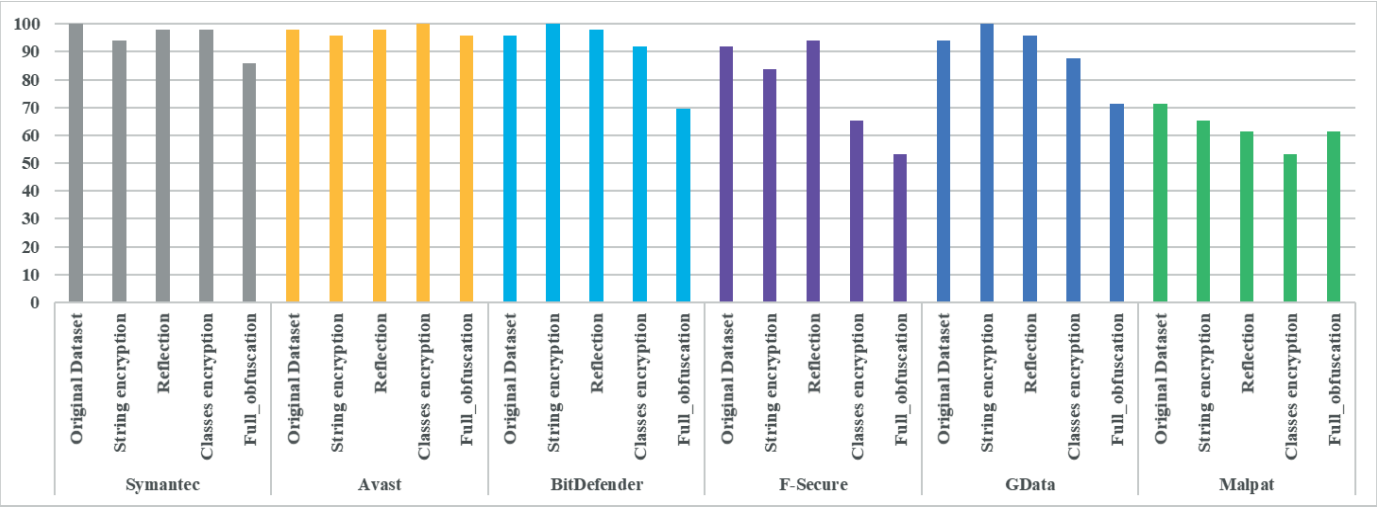


Fig. 2. Changing the used tools' detection accuracy according to obfuscation techniques

REFERENCES

1. Egham. Gartner Says Worldwide Sales of Smartphones Grew 9 Percent in First Quarter of 2017. 2017 (Accessed 30/5/2018).
2. The State of Cyber Security 2017. 2017, F-Secure Corporation.
3. Lueg, C., 8,400 new Android malware samples every day. 2017, G-Data security blog.
4. https://ibotpeaches.github.io/Apktool/. (Accessed 30/5/2018).

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:58:34 UTC from IEEE Xplore. Restrictions apply.

5. Talha, K.A., D.I. Alper, and C. Aydin, APK Auditor: Permission-based Android malware detection system. Digital Investigation, 2015. 13: p. 1-14.
6. Barrera, D., et al., A Methodology for Empirical Analysis of Permission-Based Security Models and its Application to Android. 2010. 73-84.
7. Zhang, M., Yue Duan, Heng Yin, and Zhiruo Zhao, Semantics-aware android malware classification using weighted contextual api dependency graphs, in Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security. ACM. 2014.
8. Gibler, C., et al. AndroidLeaks: automatically detecting potential privacy leaks in android applications on a large scale. in International Conference on Trust and Trustworthy Computing. 2012. Springer.
9. Moonsamy, V., J. Rong, and S. Liu, Mining permission patterns for contrasting clean and malicious android applications. Future Generation Computer Systems, 2014. 36: p. 122-132.
10. Samra, A.A.A. and O.A. Ghanem, Analysis of Clustering Technique in Android Malware Detection. 2013: p. 729-733.
11. Sanz, B., et al. On the automatic categorisation of android applications. in Consumer Communications and Networking Conference (CCNC), 2012 IEEE. 2012. IEEE.
12. Feizollah, A., et al., AndroDialysis: Analysis of Android Intent Effectiveness in Malware Detection. Computers & Security, 2017. 65: p. 121-134.
13. Wang, X., et al., Characterizing Android apps' behavior for effective detection of malapps at large scale. Future Generation Computer Systems, 2017. 75: p. 30-45.
14. Arp, D., et al. DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket. in Ndss. 2014.
15. Moghaddam, S.H. and M. Abbaspour. Sensitivity analysis of static features for Android malware detection. in Electrical Engineering (ICEE), 2014 22nd Iranian Conference on. 2014. IEEE.
16. Martín, A., H.D. Menéndez, and D. Camacho, MOCDroid: multi-objective evolutionary classifier for Android malware detection. Soft Computing, 2016. 21(24): p. 7405-7415.
17. Wu, D.-J., et al. Droidmat: Android malware detection through manifest and api calls tracing. in Information Security (Asia JCIS), 2012 Seventh Asia Joint Conference on. 2012. IEEE.
18. Aafer, Y., Wenliang Du, and Heng Yin, Droidapiminer: Mining api-level features for robust malware detection in android, in International conference on security and privacy in communication systems. p. pp. 86-103.
19. Sen, S., A.I. Aysan, and J.A. Clark. SAFEDroid: Using Structural Features for Detecting Android Malwares. in Security and Privacy in Communication Networks. 2018. Cham: Springer International Publishing.
20. Yerima, S.Y. and S. Sezer, DroidFusion: A Novel Multilevel Classifier Fusion Approach for Android Malware Detection. IEEE Transactions on Cybernetics, 2018: p. 1-14.
21. Chen, S., et al., Automated poisoning attacks and defenses in malware detection systems: An adversarial machine learning approach. Computers & Security, 2018. 73: p. 326-344.
22. Park, W., et al. Analyzing and detecting method of android malware via disassembling and visualization. in Information and Communication Technology Convergence (ICTC), 2014 International Conference on. 2014. IEEE.
23. Palumbo, P., et al., A pragmatic android malware detection procedure. Computers & Security, 2017. 70: p. 689-701.
24. Lee, J., S. Lee, and H. Lee, Screening smartphone applications using malware family signatures. Computers & Security, 2015. 52: p. 234-249.
25. Backes, M., et al., AppGuard — Fine-Grained Policy Enforcement for Untrusted Android Applications, in Revised Selected Papers of the 8th International Workshop on Data Privacy Management and Autonomous Spontaneous Security - Volume 8247. 2014, Springer-Verlag. p. 213-231.
26. Kwon, J., et al. Droidgraph: discovering android malware by analyzing semantic behavior. in Communications and Network Security (CNS), 2014 IEEE Conference on. 2014. IEEE.
27. Alam, S., et al., DroidNative: Automating and optimizing detection of Android native code malware variants. Computers & Security, 2017. 65: p. 230-246.
28. Elish, K.O., et al., Profiling user-trigger dependence for Android malware detection. Computers & Security, 2015. 49: p. 255-273.
29. Junaid, M., D. Liu, and D. Kung, Dexteroid: Detecting malicious behaviors in Android apps using reverse-engineered life cycle models. Computers & Security, 2016. 59: p. 92-117.
30. Brown, J., M. Anwar, and G. Dozier, Detection of Mobile Malware: An Artificial Immunity Approach. 2016: p. 74-80.
31. Canfora, G., et al., Effectiveness of Opcode ngrams for Detection of Multi Family Android Malware. 2015: p. 333-340.
32. Gurulian, I., et al., You can't touch this: Consumer-centric android application repackaging detection. Future Generation Computer Systems, 2016. 65: p. 1-9.
33. Aafer, Y., W. Du, and H. Yin. DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android. in Security and Privacy in Communication Networks. 2013. Cham: Springer International Publishing.
34. Feldman, S., D. Stadther, and B. Wang, Manilyzer: Automated Android Malware Detection through Manifest Analysis. 2014: p. 767-772.
35. Sheen, S., R. Anitha, and V. Natarajan, Android based malware detection using a multifeature collaborative decision fusion approach. Neurocomputing, 2015. 151: p. 905-912.
36. Du, Y., J. Wang, and Q. Li, An Android Malware Detection Approach Using Community Structures of Weighted Function Call Graphs. IEEE Access, 2017. 5: p. 17478-17486.
37. Milosevic, N., A. Dehghantanha, and K.-K.R. Choo, Machine learning aided Android malware classification. Computers & Electrical Engineering, 2017. 61: p. 266-274.
38. Verma, S. and S.K. Muttoo, An Android Malware Detection Framework-based on Permissions and Intents. Defence Science Journal, 2016. 66(6): p. 618.
39. Zheng, M., P.P. Lee, and J.C. Lui. ADAM: an automatic and extensible platform to stress test android anti-virus systems. in International conference on detection of intrusions and malware, and vulnerability assessment. 2012. Springer.
40. Faruki, P., et al., Android Security: A Survey of Issues, Malware Penetration, and Defenses. IEEE Communications Surveys & Tutorials, 2015. 17(2): p. 998-1022.
41. Maiorca, D., et al., Stealth attacks: An extended insight into the obfuscation effects on Android malware. Computers & Security, 2015. 51: p. 16-31.
42. Lantz, P. and B. Johansson. Towards bridging the gap between dalvik bytecode and native code during static analysis of android applications. in Wireless Communications and Mobile Computing Conference (IWCMC), 2015 International. 2015. IEEE.
43. Tao, G., et al., MalPat: Mining Patterns of Malicious and Benign Android Apps via Permission-Related APIs. IEEE Transactions on Reliability, 2018. 67(1): p. 355-369.
44. Zhu, H.-J., et al., DroidDet: Effective and robust detection of android malware using static analysis along with rotation forest model. Neurocomputing, 2018. 272: p. 638-646.
45. Wang, C., et al., Research on data mining of permissions mode for Android malware detection. Cluster Computing, 2018.
46. Wang, W., M. Zhao, and J. Wang, Effective android malware detection with a hybrid model based on deep autoencoder and convolutional neural network. Journal of Ambient Intelligence and Humanized Computing, 2018.
47. Abubaker, H., S.M. Shamsuddin, and A. Ali, Analytics on Malicious Android Applications. International Journal of Advances in Soft Computing & Its Applications, 2018. 10(1).
48. Kirubavathi, G. and R. Anitha, Structural analysis and detection of android botnets using machine learning techniques. International Journal of Information Security, 2017. 17(2): p. 153-167.
49. Fan, M., et al., Android Malware Familial Classification and Representative Sample Selection via Frequent Subgraph Analysis. IEEE Transactions on Information Forensics and Security, 2018. 13(8): p. 1890-1905.
50. Idrees, F., et al., PIndroid: A novel Android malware detection system using ensemble learning methods. Computers & Security, 2017. 68: p. 36-46.
51. Sokolova, K., C. Perez, and M. Lemercier, Android application classification and anomaly detection with graph-based permission patterns. Decision Support Systems, 2017. 93: p. 62-76.
52. Yang, X., et al., Characterizing malicious Android apps by mining topic-specific data flow signatures. Information and Software Technology, 2017. 90: p. 27-39.
53. Karbab, E.B., et al., Android Malware Detection using Deep Learning on API Method Sequences. arXiv preprint arXiv:1712.08996, 2017.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:58:34 UTC from IEEE Xplore. Restrictions apply.

54. Ma, L., et al., Ultra-Lightweight Malware Detection of Android Using 2-Level Machine Learning. 2016: p. 729-733.
55. Fereidooni, H., et al. ANASTASIA: ANdroid mAlware detection using STatic analySIs of Applications. in New Technologies, Mobility and Security (NTMS), 2016 8th IFIP International Conference on. 2016. IEEE.
56. Coronado-De-Alba, L.D., A. Rodríguez-Mota, and P.J. Escamilla-Ambrosio. Feature selection and ensemble of classifiers for Android malware detection. in Communications (LATINCOM), 2016 8th IEEE Latin-American Conference on. 2016. IEEE.
57. Goyal, R., et al., SafeDroid: A Distributed Malware Detection Service for Android. 2016: p. 59-66.
58. Song, J., et al., An integrated static detection and analysis framework for android. Pervasive and Mobile Computing, 2016. 32: p. 15-25.
59. Mariconti, E., et al., Mamadroid: Detecting android malware by building markov chains of behavioral models. arXiv preprint arXiv:1612.04433, 2016.
60. Wu, S., et al., Effective detection of android malware based on the usage of data flow APIs and machine learning. Information and Software Technology, 2016. 75: p. 17-25.
61. Yerima, S.Y., I. Muttik, and S. Sezer, High accuracy android malware detection using ensemble learning. IET Information Security, 2015. 9(6): p. 313-320.
62. Li, Q. and X. Li, Android Malware Detection Based on Static Analysis of Characteristic Tree. 2015: p. 84-91.
63. Li, Q. and X. Li. Android malware detection based on static analysis of characteristic tree. in Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC), 2015 International Conference on. 2015. IEEE.
64. Li, W., J. Ge, and G. Dai. Detecting malware for android platform: An svm-based approach. in Cyber Security and Cloud Computing (CSCloud), 2015 IEEE 2nd International Conference on. 2015. IEEE.
65. Gordon, M.I., et al., Information-Flow Analysis of Android Applications in DroidSafe. 2015.
66. Suarez-Tangil, G., et al., Dendroid: A text mining approach to analyzing and classifying code structures in Android malware families. Expert Systems with Applications, 2014. 41(4): p. 1104-1117.
67. Zhang, M., et al., Semantics-Aware Android Malware Classification Using Weighted Contextual API Dependency Graphs. 2014: p. 1105-1116.
68. Yerima, S.Y., G. McWilliams, and S. Sezer, Analysis of Bayesian classification-based approaches for Android malware detection. IET Information Security, 2014. 8(1): p. 25-36.
69. Feng, Y., et al. Apposcopy: Semantics-based detection of android malware through static analysis. in Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. 2014. ACM.
70. Xiaoyan, Z., F. Juan, and W. Xiujuan, Android malware detection based on permissions. 2014.
71. Liang, S. and X. Du. Permission-combination-based scheme for android mobile malware detection. in Communications (ICC), 2014 IEEE International Conference on. 2014. IEEE.
72. Hu, W., et al. Migdroid: Detecting app-repackaging android malware via method invocation graph. in Computer Communication and Networks (ICCCN), 2014 23rd International Conference on. 2014. IEEE.
73. Idrees, F. and M. Rajarajan. Investigating the android intents and permissions for malware detection. in Wireless and Mobile Computing, Networking and Communications (WiMob), 2014 IEEE 10th International Conference on. 2014. IEEE.
74. Raphael, R., P. Vinod, and B. Omman. X-ANOVA and X-Utest features for Android malware analysis. in Advances in Computing, Communications and Informatics (ICACCI, 2014 International Conference on. 2014. IEEE.
75. Shen, T., et al., Detect Android Malware Variants Using Component Based Topology Graph. 2014: p. 406-413.
76. Zhang, L., Yan Niu, Xiao Wu, Zhaoguo Wang, and Yibo A3: Automatic Analysis of Android Malware, in International Workshop on Cloud Computing and Information Security. 2013.
77. Aung, Z. and W. Zaw, Permission-based android malware detection. International Journal of Scientific & Technology Research, 2013. 2(3): p. 228-234.
78. Faruki, P., et al., AndroSimilar: Robust signature for detecting variants of Android malware. Journal of Information Security and Applications, 2015. 22: p. 66-80.
79. Peiravian, N. and X. Zhu, Machine Learning for Android Malware Detection Using Permission and API Calls. 2013: p. 300-305.
80. Glodek, W. and R. Harang, Rapid Permissions-Based Detection and Analysis of Mobile Malware Using Random Decision Forests. 2013: p. 980-985.
81. Lu, Y., et al., Android Malware Detection Technology Based on Improved Bayesian Classification. 2013: p. 1338-1341.
82. Crussell, J., C. Gibler, and H. Chen, AnDarwin: Scalable Detection of Android Application Clones Based on Semantics. IEEE Transactions on Mobile Computing, 2015. 14(10): p. 2007-2019.
83. Zhou, W., et al. Detecting repackaged smartphone applications in third-party android marketplaces. in Proceedings of the second ACM conference on Data and Application Security and Privacy. 2012. ACM.
84. Grace, M., et al., RiskRanker: Scalable and Accurate Zero-day Android. 2012.
85. Lin, J., X. Zhao, and H. Li, Target: category-based android malware detection revisited, in Proceedings of the Australasian Computer Science Week Multiconference. 2017, ACM: Geelong, Australia. p. 1-9.
86. Felt, A.P., et al., Android permissions demystified, in Proceedings of the 18th ACM conference on Computer and communications security. 2011, ACM: Chicago, Illinois, USA. p. 627-638.
87. Chin, E., et al., Analyzing inter-application communication in Android, in Proceedings of the 9th international conference on Mobile systems, applications, and services. 2011, ACM: Bethesda, Maryland, USA. p. 239-252.
88. Enck, W., M. Ongtang, and P. McDaniel, On lightweight mobile phone application certification, in Proceedings of the 16th ACM conference on Computer and communications security. 2009, ACM: Chicago, Illinois, USA. p. 235-245.
89. Zhou, Y. and X. Jiang, Dissecting Android Malware: Characterization and Evolution. 2012: p. 95-109.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:58:34 UTC from IEEE Xplore. Restrictions apply.